
Mini Project Report: LIBRARY MANAGEMENT SYSTEM

People's Democratic Republic of Algeria Ministry of Higher Education
and Scientific Research



University of 20 August 1955-Skikda
Faculty of Sciences
Department of Computer Science



Subject

Mini Project Report LIBRARY MANAGEMENT SYSTEM

Presented by
ABD EL WAHED Ihab
& SAIEB Ayoub

1. PROJECT DESCRIPTION

The Library Management System is a complete backend and frontend application developed using Flask, SQLite, JWT authentication, and Streamlit.

The project allows administrators to manage books in the library and allows users to browse, borrow, or purchase books.

The system was designed using a clean architecture based on routes, services, database layer, authentication module, and utility functions.

The project also includes:

- REST API
- JWT Authentication
- Console Application
- Streamlit Web Interface
- SQLite Database

2. PROJECT OBJECTIVES

The objectives of this project are:

- Learn REST API development using Flask
- Apply software architecture concepts
- Implement JWT authentication
- Use SQLite database
- Build CRUD operations
- Separate business logic from routes
- Create a professional frontend using Streamlit
- Create a console application for testing

3. TECHNOLOGIES USED

Technology	Purpose
Python	Main programming language
Flask	REST API backend
SQLite	Database
Flask-JWT-Extended	Authentication
Streamlit	Frontend interface
Requests	API communication
Postman	API testing

4. PROJECT ARCHITECTURE

The project follows a layered architecture:

- Routes Layer:

Handles API endpoints and HTTP requests.

- Services Layer:

Contains the business logic of the application.

- Database Layer:

Handles SQLite database connection and table creation.

- Authentication Layer:

Manages JWT authentication and authorization.

- Utils Layer:

Contains validation functions and reusable utilities.

- Frontend Layer:

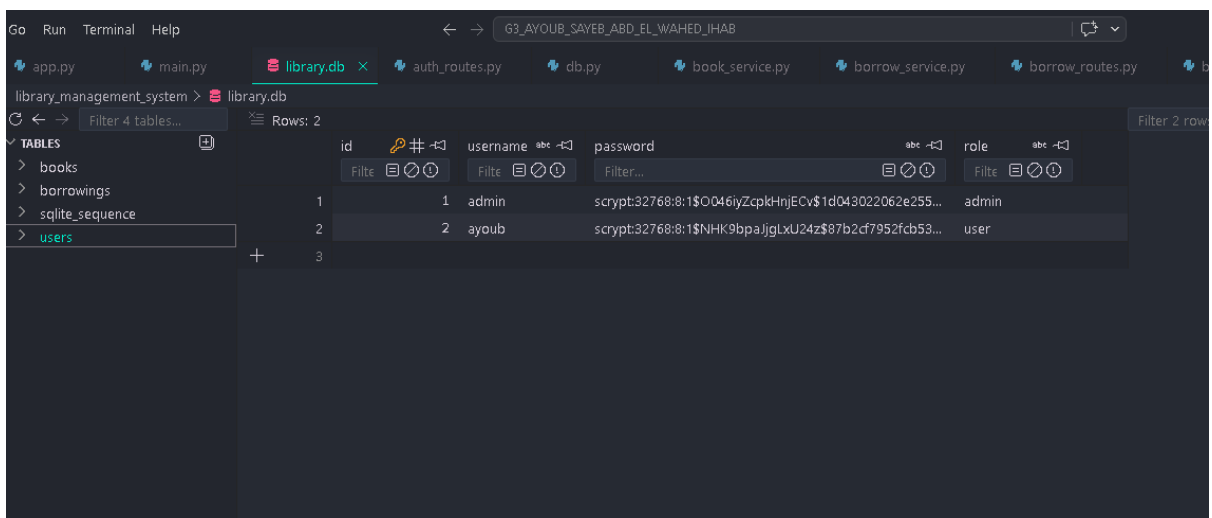
Implemented using Streamlit for user interaction.

5. PROJECT STRUCTURE

```
library_management_system/  
├── app.py  
├── database/  
├── routes/  
├── services/  
├── auth/  
├── utils/  
├── models/  
├── console_app/  
└── streamlit_app/
```

6. DATABASE DESIGN

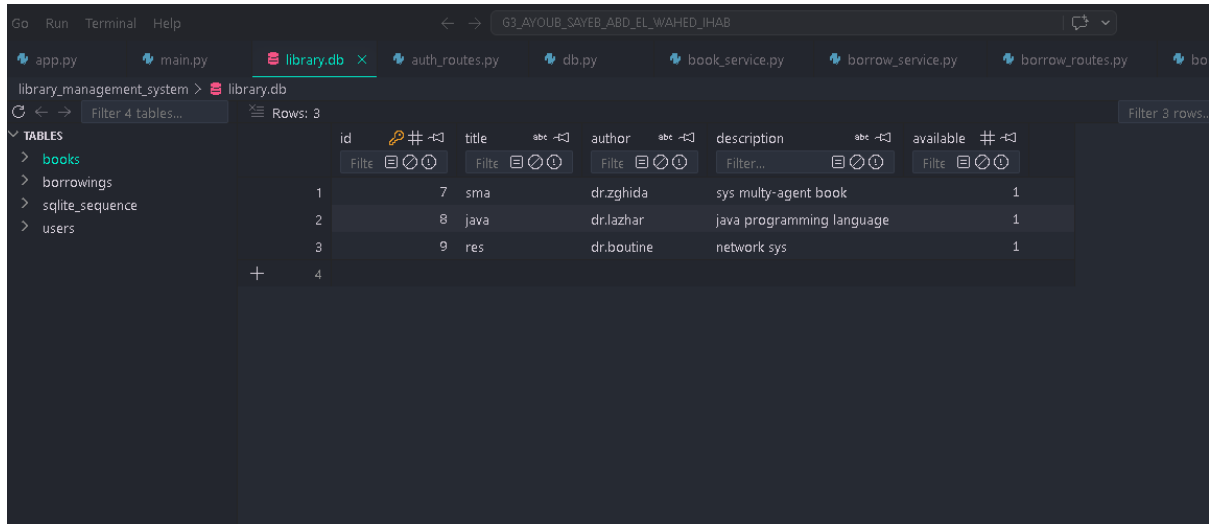
USERS TABLE:



id	username	password	role
1	admin	script:32768:8:1\$O046iyZcpkHnjECv\$1d043022062e255...	admin
2	ayoub	script:32768:8:1\$NHK9bpaJjgLU24z\$87b2cf7952fcb53...	user

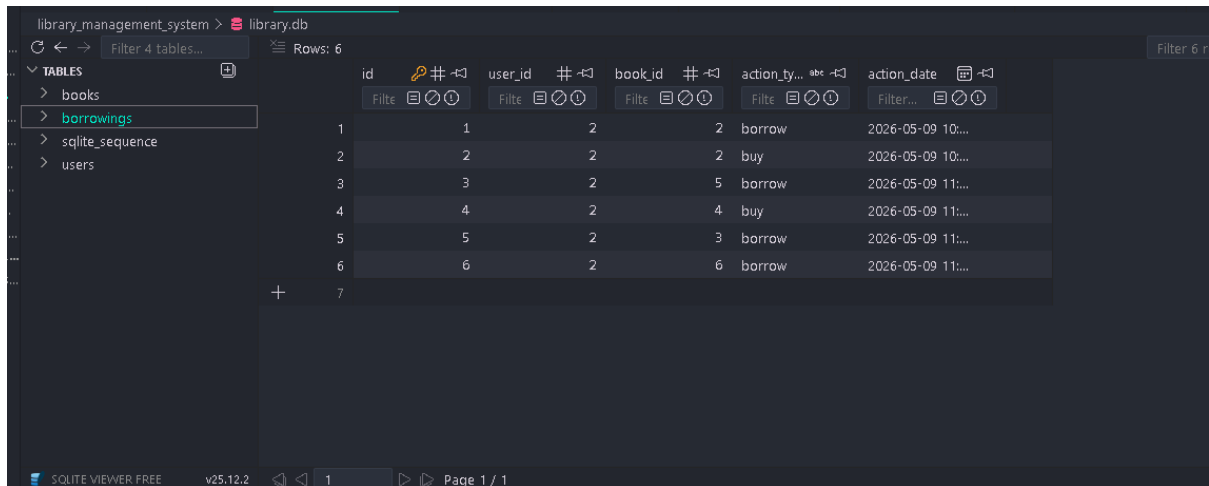
Mini Project Report: LIBRARY MANAGEMENT SYSTEM

BOOKS TABLE:



id	title	author	description	available
7	sma	dr.zghida	sys multy-agent book	1
8	java	dr.lazhar	java programming language	1
9	res	dr.boutine	network sys	1

BORROWINGS TABLE:



id	user_id	book_id	action_ty	action_date
1	1	2	borrow	2026-05-09 10:...
2	2	2	buy	2026-05-09 10:...
3	3	2	borrow	2026-05-09 11:...
4	4	2	buy	2026-05-09 11:...
5	5	2	borrow	2026-05-09 11:...
6	6	2	borrow	2026-05-09 11:...

7. API ENDPOINTS

AUTH ROUTES :

Method	Endpoint	Description
POST	/auth/register	Register user
POST	/auth/login	Login user

BOOK ROUTES :

Method	Endpoint	Description
GET	/books/	Get all books
POST	/books/	Add book
PUT	/books/{id}	Update book
DELETE	/books/delete-by-name/{title}	Delete book

BORROW ROUTES :

Method	Endpoint	Description
POST	/borrow/borrow/{id}	Borrow book
POST	/borrow/buy/{id}	Buy book

8. AUTHENTICATION SYSTEM

The application uses JWT authentication, When the user logs in successfully, the server generates a JWT token, The token is then used to access protected routes.

Admin users can:

- Add books
- Update books
- Delete books

Normal users can:

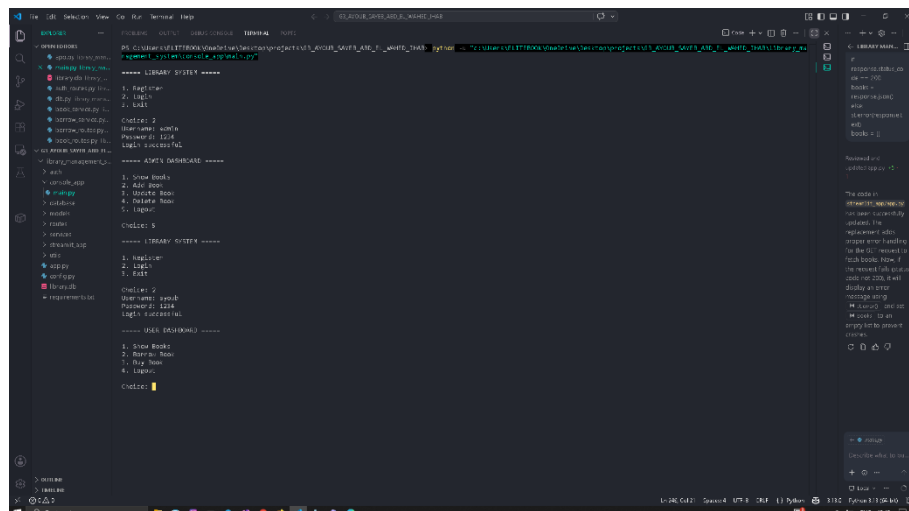
- View books
- Borrow books
- Buy books

9. CONSOLE APPLICATION

A console application was developed to test the backend system.

The console application allows:

- Registration
- Login
- Book visualization
- Borrowing books
- Buying books and Admin management operations

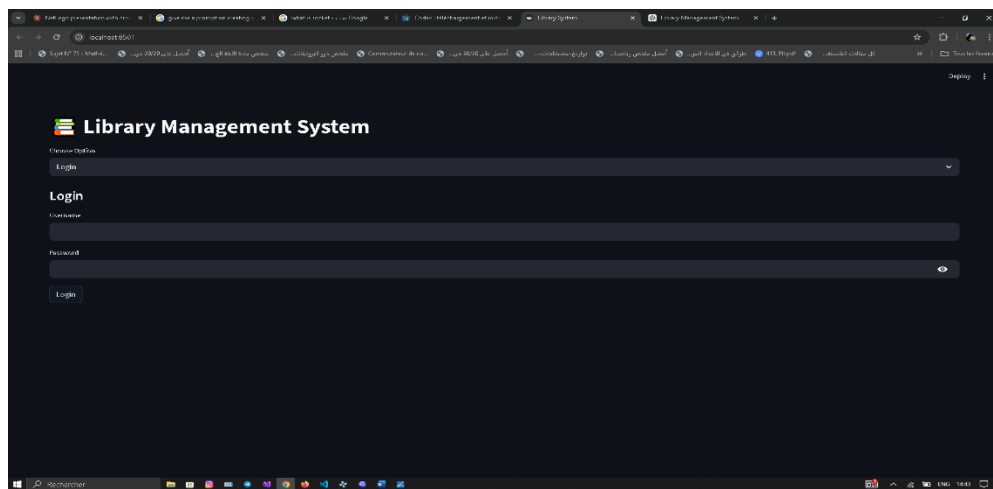


10. STREAMLIT FRONTEND

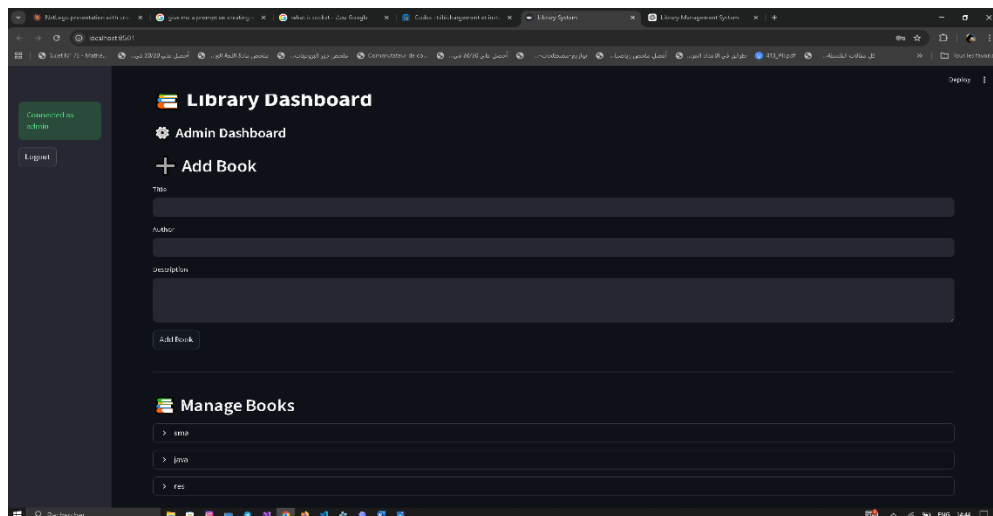
A professional frontend interface was developed using Streamlit.

Features:

- Login/Register page
- Admin dashboard
- User dashboard
- Book management
- Borrow and buy functionalities
- Real-time database updates

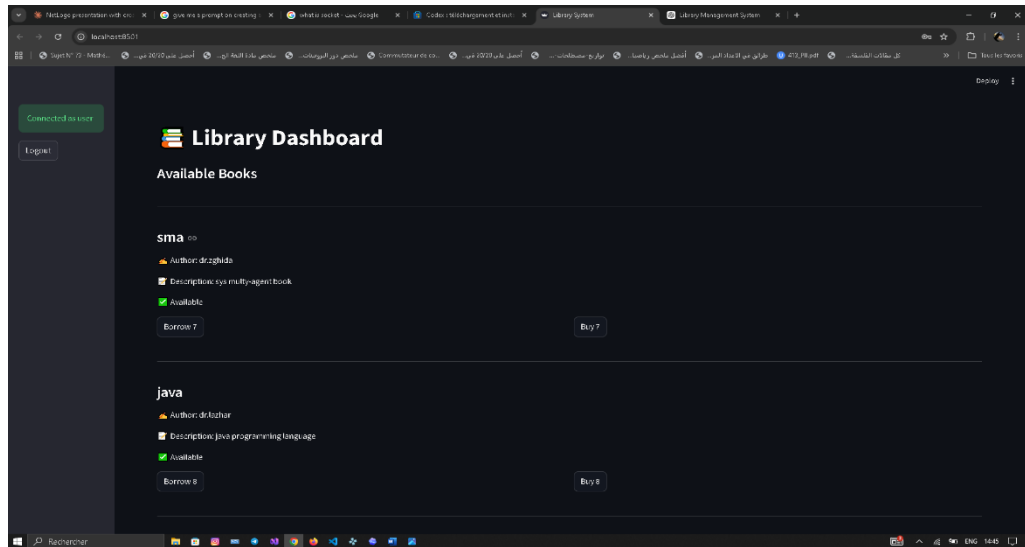


Login page

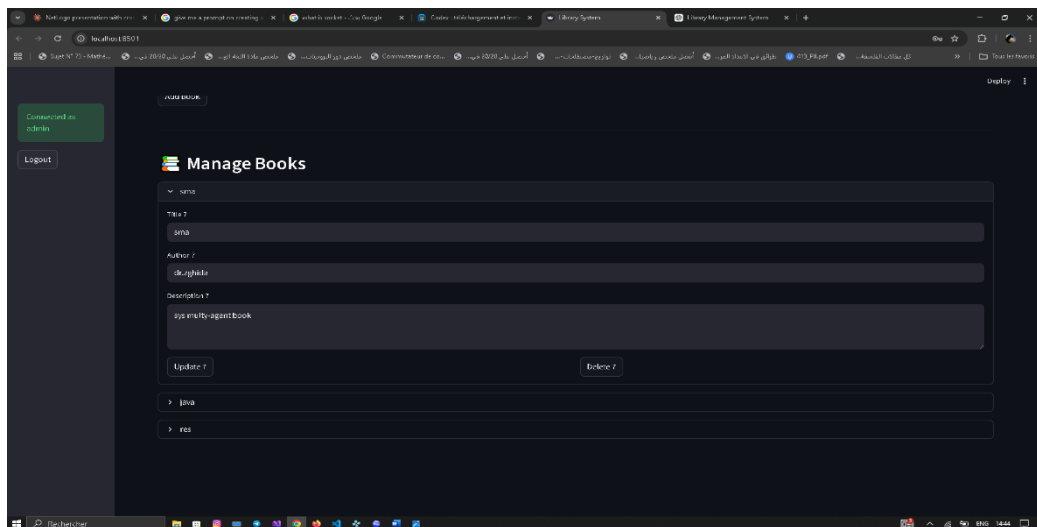


Admin view

Mini Project Report: LIBRARY MANAGEMENT SYSTEM



User view



Book management view (update-delete)

11. HOW TO RUN THE PROJECT

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Run Flask backend:

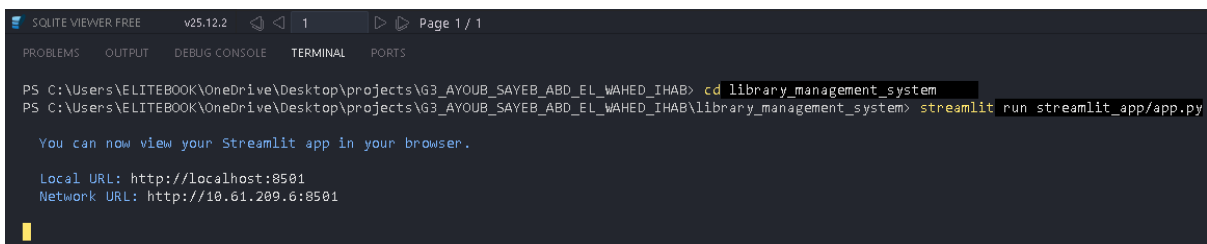
```
python app.py
```

3. Run Streamlit frontend:

```
streamlit run streamlit_app/app.py
```

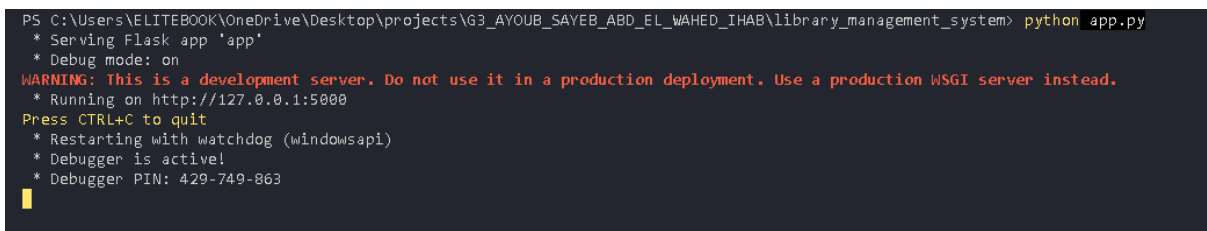
4. Run console application:

```
python console_app/main.py
```



A screenshot of a Windows terminal window. The title bar shows 'SQLITE VIEWER FREE v25.12.2'. The terminal has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The command prompt shows the user is in the directory 'C:\Users\ELITEBOOK\OneDrive\Desktop\projects\G3_AYOUB_SAYEB_ABD_EL_WAHED_IHAB\library_management_system'. The command 'streamlit run streamlit_app/app.py' has been executed. The output shows the local and network URLs for the Streamlit app: 'Local URL: http://localhost:8501' and 'Network URL: http://10.61.209.6:8501'.

streamlit running



A screenshot of a Windows terminal window. The title bar shows 'SQLITE VIEWER FREE v25.12.2'. The terminal has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The command prompt shows the user is in the directory 'C:\Users\ELITEBOOK\OneDrive\Desktop\projects\G3_AYOUB_SAYEB_ABD_EL_WAHED_IHAB\library_management_system'. The command 'python app.py' has been executed. The output shows the Flask app is running on 'http://127.0.0.1:5000'. It also displays a warning: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.' and other information like 'Press CTRL+C to quit', 'Restarting with watchdog (windowsapi)', 'Debugger is active!', and 'Debugger PIN: 429-749-863'.

flask-app running

Mini Project Report: LIBRARY MANAGEMENT SYSTEM

The screenshot displays the Visual Studio Code (VS Code) interface for a project named "library_management_system". The Explorer panel on the left shows the project structure, including files like `app.py`, `main.py`, `library.db`, `auth_routes.py`, `db.py`, `book_service.py`, `borrow_service.py`, `borrow_routes.py`, and `book_routes.py`. The main editor displays the `library.db` file, which contains a SQLite database with tables: `books`, `borrowings`, `sqlite_sequence`, and `users`. The `users` table is expanded, showing two rows:

id	username	password
1	admin	scrypt:32768
2	ayoub	scrypt:32768

The bottom panel shows the TERMINAL output, displaying a series of HTTP requests and responses, including:

- `GET /books/ HTTP`
- `PUT /books/1 HTTP`
- `DELETE /books/delete-by-name/SMA HTTP`
- `POST /auth/login HTTP`

CONCLUSION

This project allowed us to understand and apply software architecture concepts using Flask and SQLite.

We implemented:

- REST APIs
- JWT authentication
- CRUD operations
- Database persistence
- Console and web interfaces

The project demonstrates a clean and scalable backend architecture.